Companion — animATEM user guide

This module drives animATEM from a Stream Deck, or any other Bitfocus Companion surface: cut, auto and fade to black per M/E, program, preview and aux selection by input name, and SuperSource/DVE memory recall, with tally.

The README covers installing the module. This guide is about building a surface with it, and what to be careful of.

> **Before you rely on this:** the module's automated tests are eight vitest cases for its WebSocket client (`src/wsClient.spec.ts` : connecting, message dispatch, sending and reconnecting, against a mock socket), and **none of them checks its messages against animATEM's**. Its protocol file is a copy of animATEM's own, kept up to date by hand (they're separate packages and can't import each other), so a change to the control server's message shapes breaks this module **silently**: a button stops working mid-show with no error anywhere obvious. Keep the app and the module in step, and re-test a page after upgrading either one.
>
> This module was built with AI assistance, directed and reviewed by a human author.

What a press actually does

This puts things on air. There's no confirmation step and no undo.

There's a second thing to understand before you build anything, because every design decision below follows from it: **animATEM's control server never replies.** You get no result and no error frame. It drops malformed messages without a word, and when a command fails it catches it and logs it *at the app* rather than passing it back.

So **a command sent while the switcher's disconnected looks just like one that worked.**

You'll run into that in two places:

- **No button here lights up from its own press.** Every feedback reads the app's snapshot instead, because "I sent it" doesn't tell you anything happened.
- **Take and aux-follow refuse rather than guess.** When there's no snapshot, `Take preview to program` and `Point an aux at program` do nothing and say so, rather than acting on a guess about what's on preview. Guess wrong and the wrong thing goes to air.

Connecting

animATEM runs its control server on `127.0.0.1:51234` whenever the app's open. There's nothing to set up on the app side.

That server has no authentication. It's only safe because it binds to loopback: anything that can reach that port can put something on air. So Companion has to run on the same machine as animATEM.

Put both connection lights on the page

Light	Answers
Connected to animATEM	Can Companion reach the app?
animATEM's ATEM connection is up	Can the app reach the switcher?

They're two different questions and they fail separately. The second goes dark on its own while a switcher reboots, with the first still green, and **while it's dark, every tally on the surface is stale rather than false.** It's showing the last thing it knew.

Mix them up and you'll restart the wrong thing at the worst possible moment. Put both on every page that has transition buttons.

Inputs are picked by name

An ATEM numbers its sources in a way that must have made sense to someone once: 1..N, then black, bars, media players and SuperSource from 3010 up. The numbers aren't contiguous and nobody remembers them. animATEM already sends the input list with short and long names, so **every input dropdown is filled from that**, and every generated bus preset is labelled with the switcher's own short name, the one the multiview shows.

There's a catch worth planning around: **bus presets only exist once a switcher has connected.** Before that the module doesn't know what inputs there are. The transition, memory and status sections work either way.

Every dropdown also takes a typed number, so a button built while animATEM was offline still works. That's the way to do it if you're building a page in advance on a laptop.

Auto goes amber mid-transition

During an auto, **both buses are on air**. So for the length of the transition a plain program tally is only telling you half the story, and that's just when someone's looking at it.

The generated Auto and PGM/PVW display buttons show that state in **amber**. Keep it. A two-colour tally during a mix is wrong for a second and a half every time.

Aux follow is a one-shot

animATEM has no follow mode. **Point an aux at what is on program** reads program *once* and sets the aux.

To make it stick, drive it from a repeating trigger. If you were expecting it to behave like a proper aux follow and it doesn't, that's why.

Memories can go stale

`memories` is the one message animATEM does **not** broadcast when it changes. It arrives on connect, and when the app explicitly asks for it.

The module rebuilds its memory dropdowns and presets whenever one arrives, but a memory added in the app without that call **won't appear until the module reconnects**. If a new memory is missing from the dropdown, disable and re-enable the connection.

Building a surface that fails safe

1. **Both connection lights, on every page with transitions.** See above.
2. **Amber for mid-transition**, from the generated presets.
3. **Program and preview rows from the generated bus presets**, once the switcher has connected. They carry the switcher's own short names, so the surface matches the multiview.
4. **Don't build a page that assumes a press worked.** Nothing is acknowledged; the feedbacks read state, and state is what you should be looking at.

Troubleshooting

Symptom	Cause
Buttons do nothing, no error	Almost the default failure here — nothing is acknowledged. Check the ATEM connection light before suspecting the module.
No bus presets at all	No switcher has connected yet, so the input list is unknown. The transition and memory sections still work.
Tally is wrong	Check whether the ATEM light is dark. While it is, every tally is the last known value.
A new memory is missing	The app did not broadcast the memory list. Reconnect the module.
Aux stops following	It never was. It is a one-shot; drive it from a trigger.
A button worked yesterday and silently stopped	The app and the module have drifted — the protocol is mirrored by hand. Check versions.

See also

- README: installing, and the full list of actions, feedbacks and variables
- `companion/HELP.md`: the same material, in Companion's help panel